
Relates to Lab 5. This is an extended module that covers TCP flow control, congestion control, and error control in TCP.

C Flow Control

What is Flow/Congestion/Error Control ?

- **Flow Control:** Algorithms to prevent that the sender overruns the receiver with information?
 - **Congestion Control:** Prevents congestion and ensures that the sender does not overwhelm the network.
 - **Error Control:** Detects and corrects errors in the data transmitted, ensuring the integrity of the data and dealing with the effects from packet losses.
- The goal of each control mechanism is different.
- But the implementation is combined

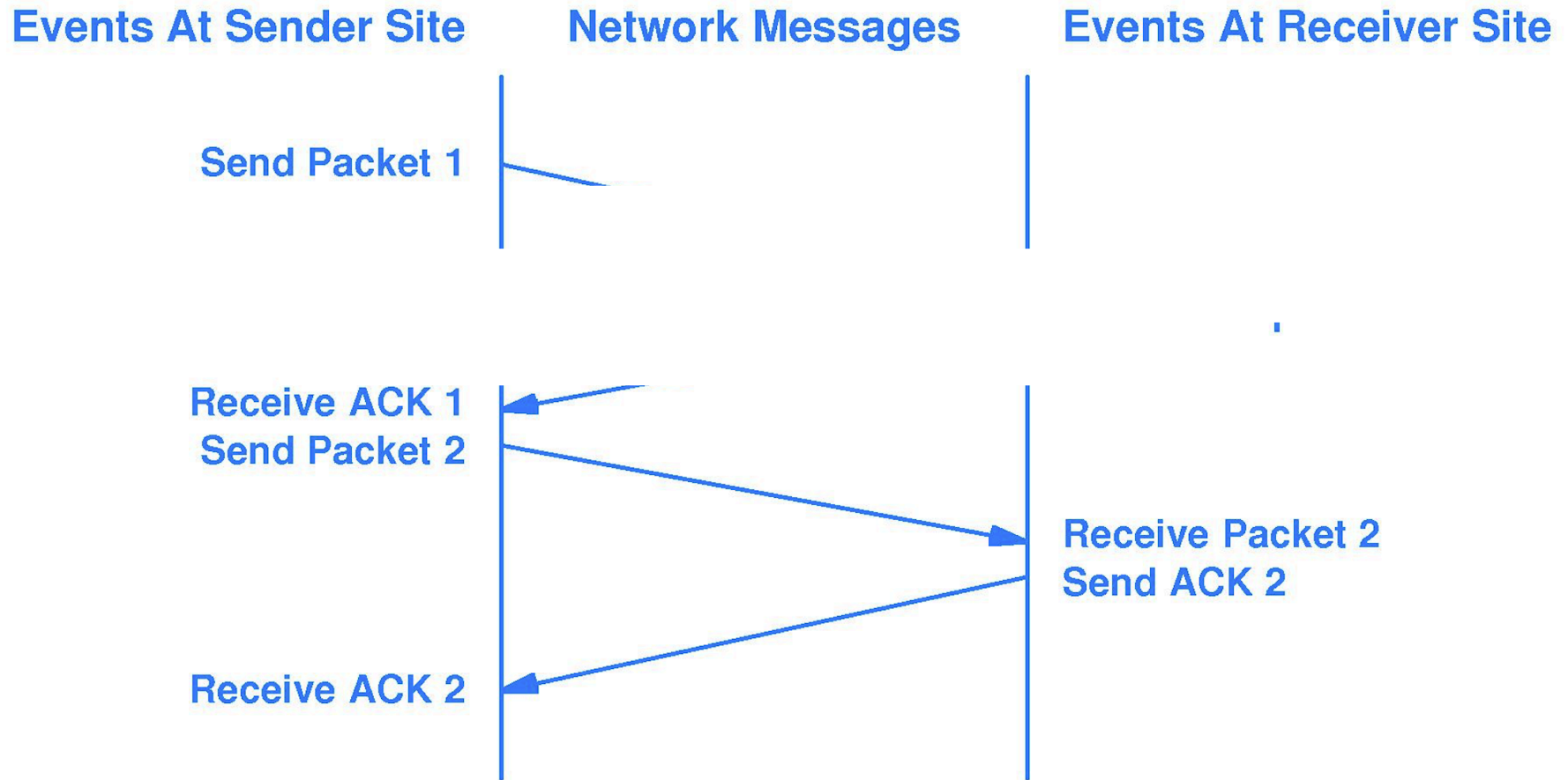
TCP -> Reliable

- Traditional technique: Positive Acknowledgement with Retransmission (PAR)

– Receiver sends when data arrives

ackno

ACKs and Retransmission – Simple Method



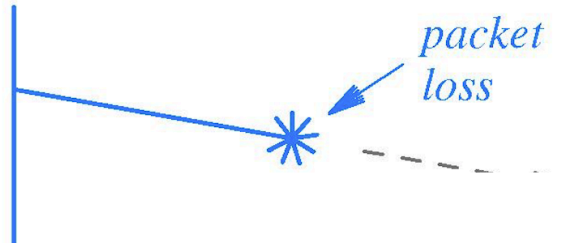
Packet Loss and Error Recovery

Events At Sender Site

Network Messages

Events At Receiver Site

Send Packet 1
Start Timer

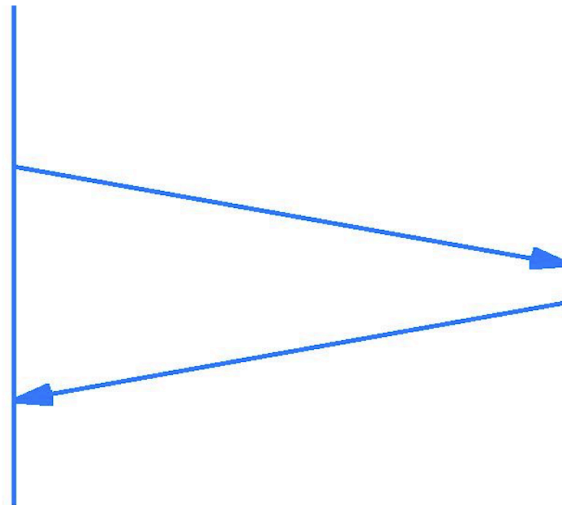


Packet should have arrived
ACK would have been sent

arrive

Timer Expires
Retransmit Packet 1
Start Timer

Receive ACK 1
Cancel Timer

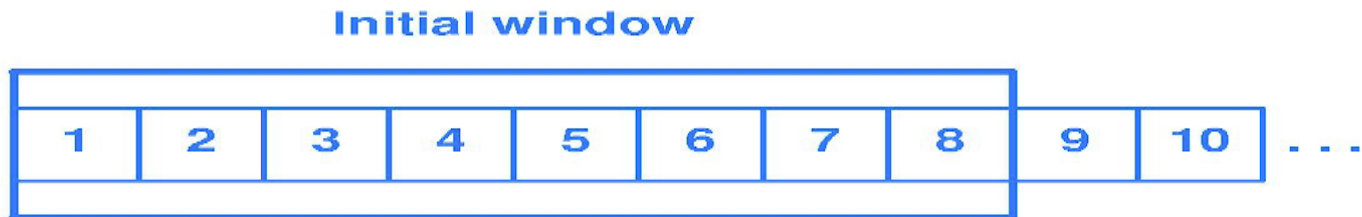


Receive Packet 1
Send ACK 1

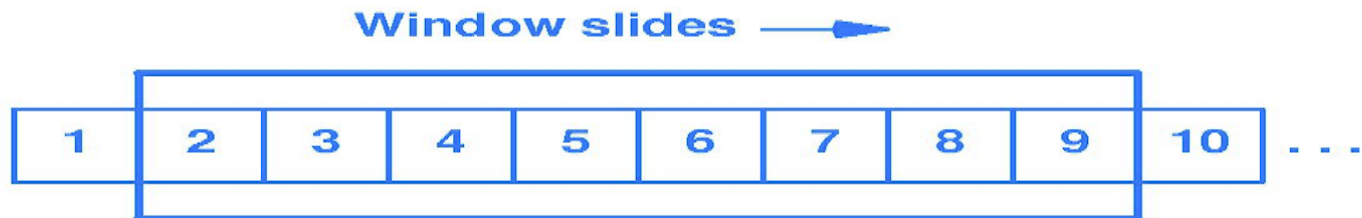
Multiple Packet Transmission – Increased Efficiency

- Allow multiple packets to be outstanding at any time
- Still require acknowledgements and retransmission
- Known as *sliding*

- When $\mathbf{A}$ is $n \times n$ and $\mathbf{b}$ is $n \times 1$, then $\mathbf{A}^{-1}\mathbf{b}$ is forward

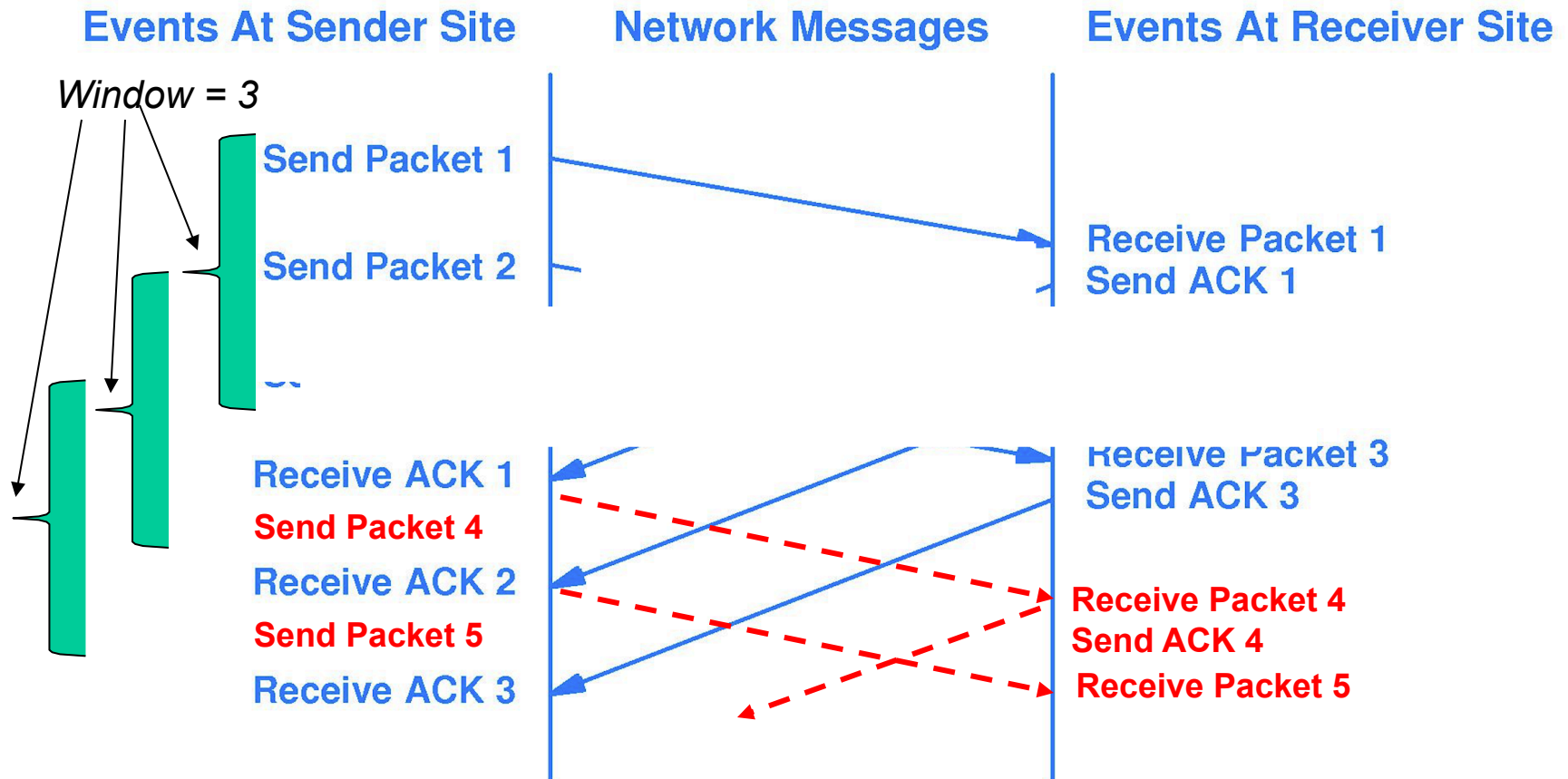


(a)



(b)

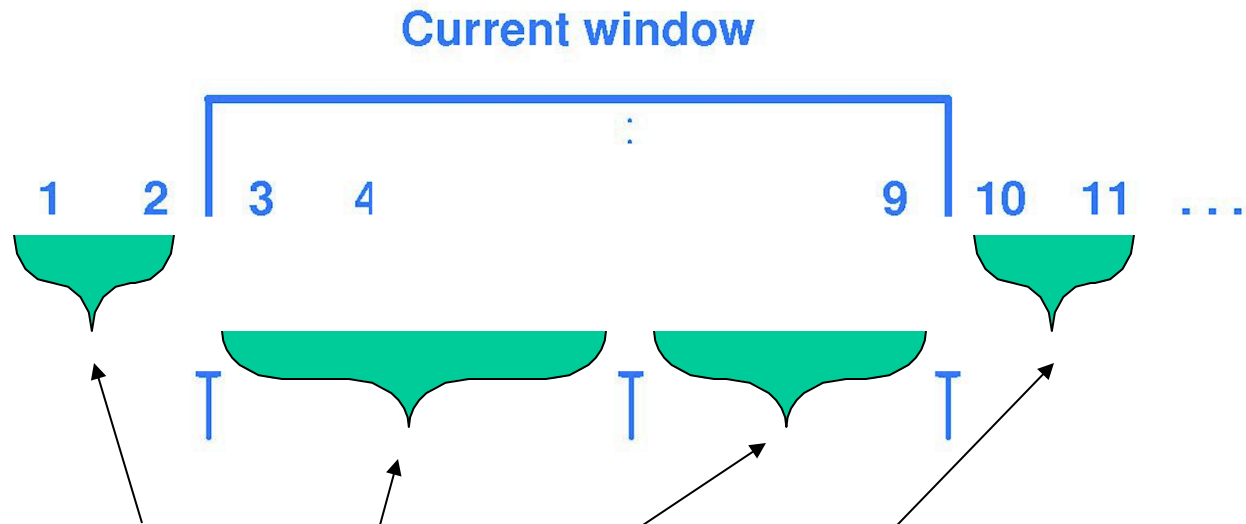
Illustration of Sliding Window



Because a well-tuned sliding window protocol keeps the network completely saturated with packets, it obtains substantially higher throughput than a simple positive acknowledgement protocol.

TCP's Sliding Window

- Measured in byte positions – window is 7bytes



- Bytes 1 through 2 are acknowledged
- Bytes 3 through 6 sent but not yet acknowledged
- Bytes 7 through 9 can be sent if in buffer
- Any bytes above 9 lie outside the window and cannot be sent



TCP Flow Control – At Receiver

TCP implements a form of sliding window flow control

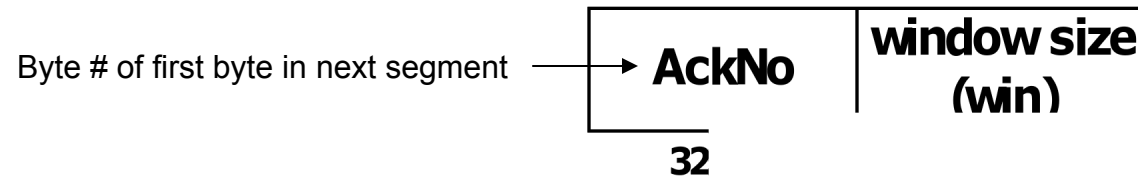
- Sending acknowledgements is **separated** from setting the window size

win

- Acknowledgements are **cumulative**

Window Management in TCP

- The receiver returns two parameters to the sender to control the flow



-

- I a

- SeqNo= AckNo**

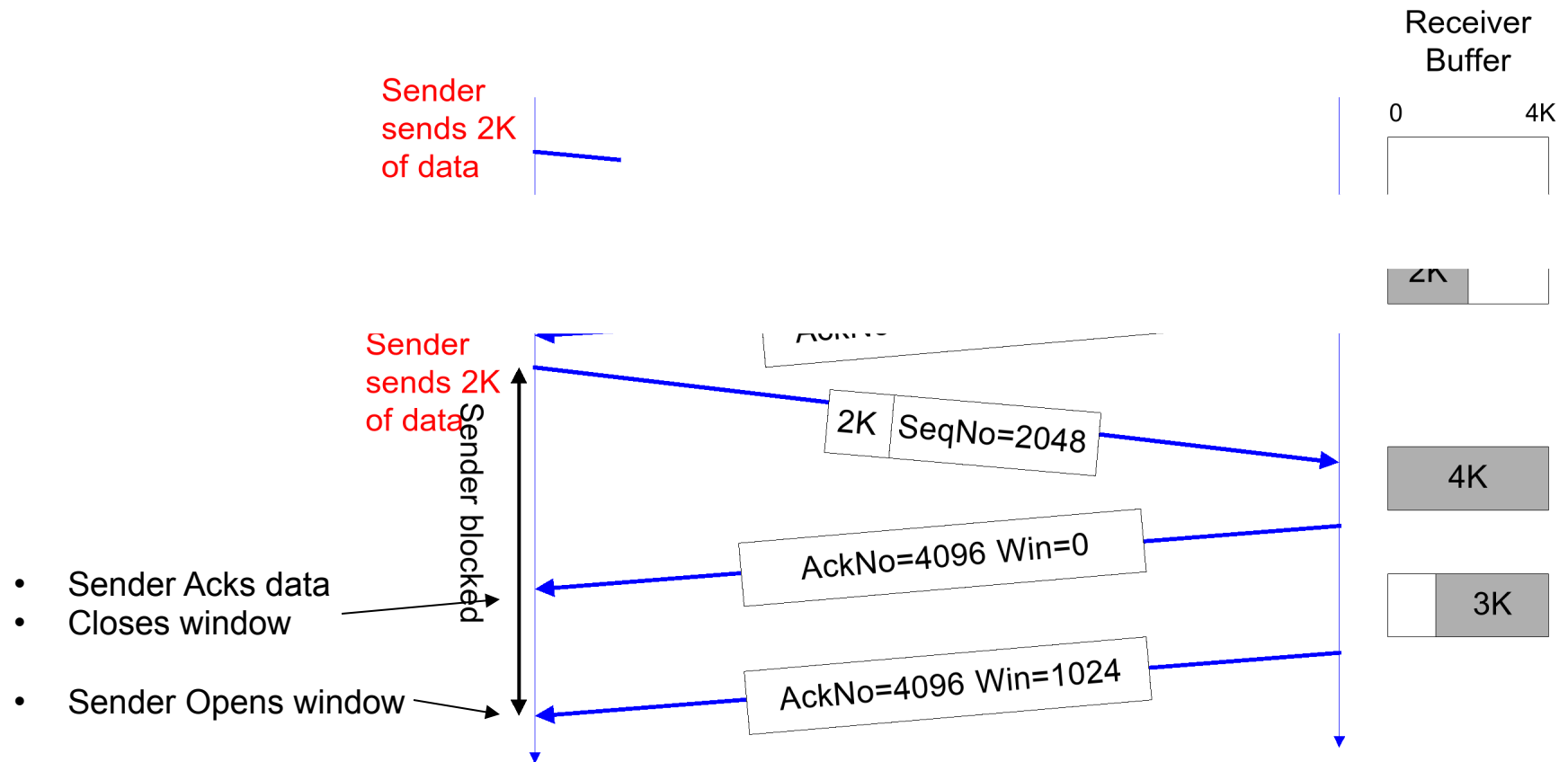
- **And number of bytes that I can receive is:**

- AckNo, AckNo+1,, AckNo+Win-1**

- With size up to Window size (win)**

- Receiver can acknowledge data **without opening the window**
- Receiver can change the window size **without acknowledging data**

Sliding Window: Example





TCP Congestion Control – Implemented at Sender to prevent overflow in the Network

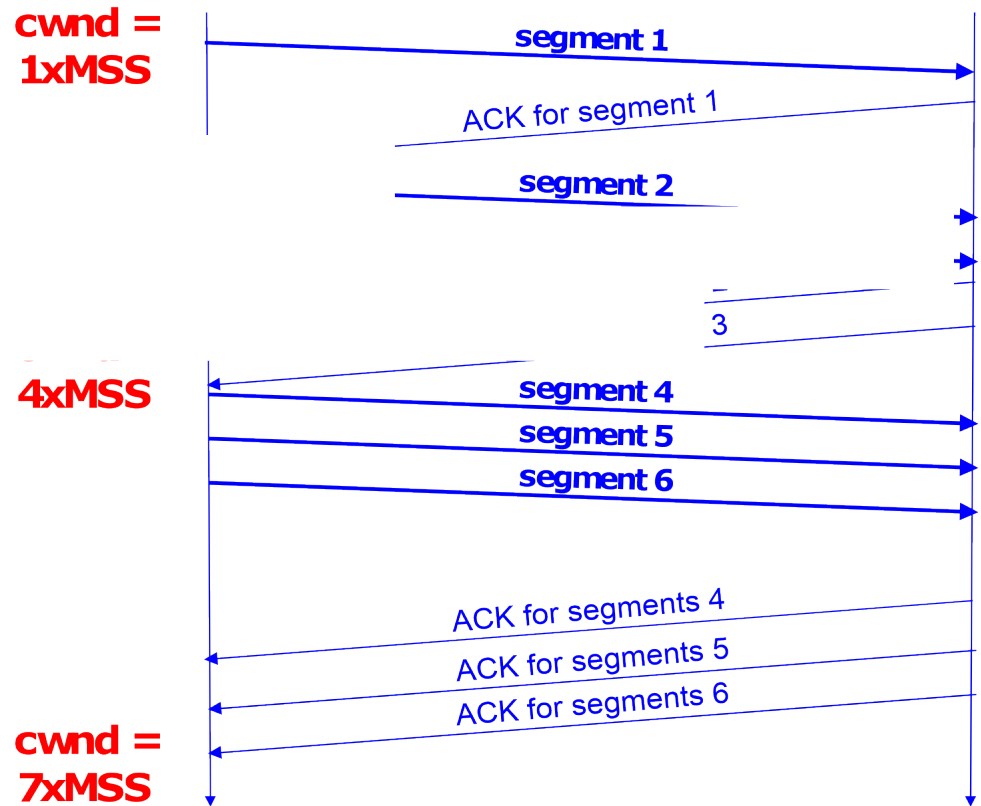
- The sender uses two parameters:
 - **Congestion Window (cwnd)**
Initial value is 1 MSS (=maximum segment size) counted in bytes
 - **Slow-start thres** **resh)**
- |
(i.e., the s
- Congestion control works in two modes:
 - **slow start** ($cwnd < ssthresh$)
 - **congestion avoidance** ($cwnd \geq ssthresh$)

Slow Start

- Initial value:
 - $\text{cwnd} = 1 \text{ segment}$
- cwnd is measured in bytes
 - $1 \text{ segment} = \text{MSS}$
- - increased
 - $\text{cwnd} = \text{cwnd} + 1 \text{ segment}$
 - If an ACK acknowledges two or more segments (cumulative ACK), cwnd is still increased by only 1 segment.
 - Even if ACK acknowledges a segment that is smaller than MSS bytes long, cwnd is still increased by 1 segment.
- Does Slow Start increment slowly? Not really.
In fact, the increase of cwnd can be exponential

Slow Start Example

- The congestion window size grows very rapidly
 - For every ACK, w increases by 1 MSS
- TCP slows down the increase of $cwnd$ and goes into congestion avoidance mode when $cwnd \geq ssthresh$



Congestion Avoidance

- Congestion avoidance phase is started if **cwnd** has reached the slow-start threshold value
- $\lfloor \frac{ssthresh + cwnd}{2} \rfloor$
 - cwnd

Where $\lfloor cwnd \rfloor$ is the largest integer smaller than cwnd
- So *cwnd* is increased by one segment (=MSS bytes) only if **all segments** have been acknowledged in the previous congestion window size.

Slow Start / Congestion Avoidance

if $cwnd \leq ssthresh$ **then**

else $/* \ c$

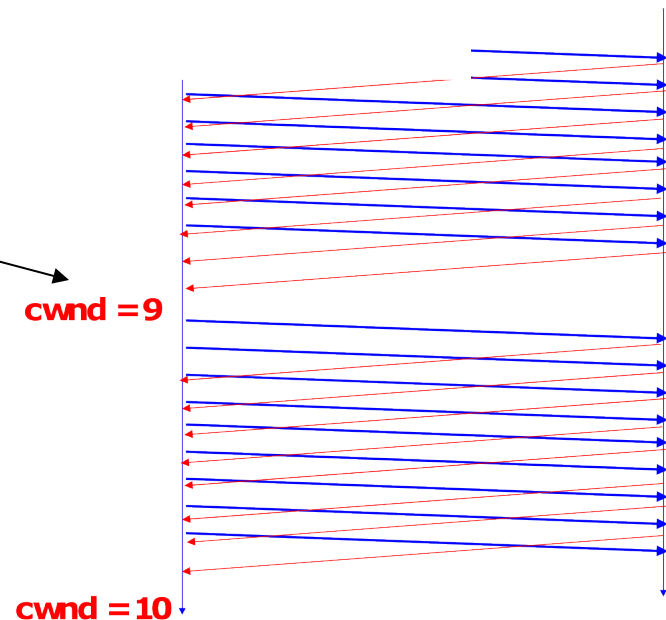
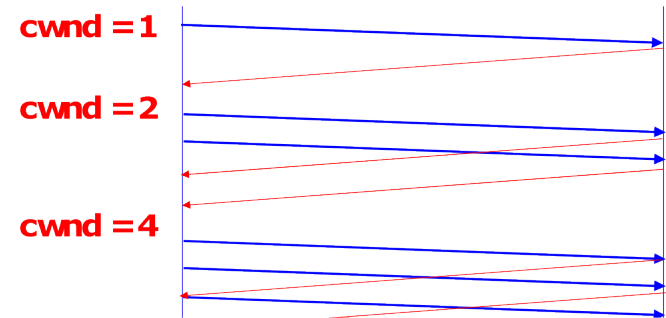
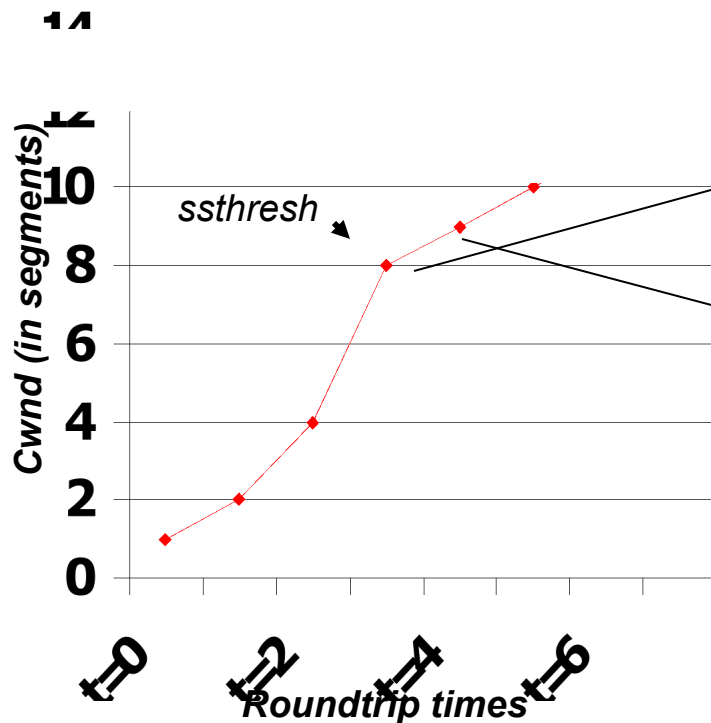
Each time an Ack is received :

$cwnd = cwnd + 1 / [cwnd]$

endif

Example of Slow Start/Congestion Avoidance

Assume that *ssthresh* = 8



TCP Detection of Congestion

- TCP tries to prevent congestion
- However congestion can still occur. SO:
- When TCP transmits a packet, it sets a timer.
- When a packet's timer expires it is due to
C
 - Bit error or a packet is not acknowledged on time, a sender assumes a loss due to buffer overflow
- Uses retransmissions as measure of congestion – when they happen frequently - > high congestion
- Reduces the congestion window (cwnd) as retransmissions increase

Detecting Congestion

- A TCP **sender** can **detect** a packet loss via:
 - Timeout of a retransmission timer
- receive out of order packet – i.e. received Seq No. $\neq$ Ack No it sent the sender in last packet.

TCP Tahoe – Classic TCP

- Congestion is assumed if sender has timed-out or received a duplicate ACK
- Each time when con

CW

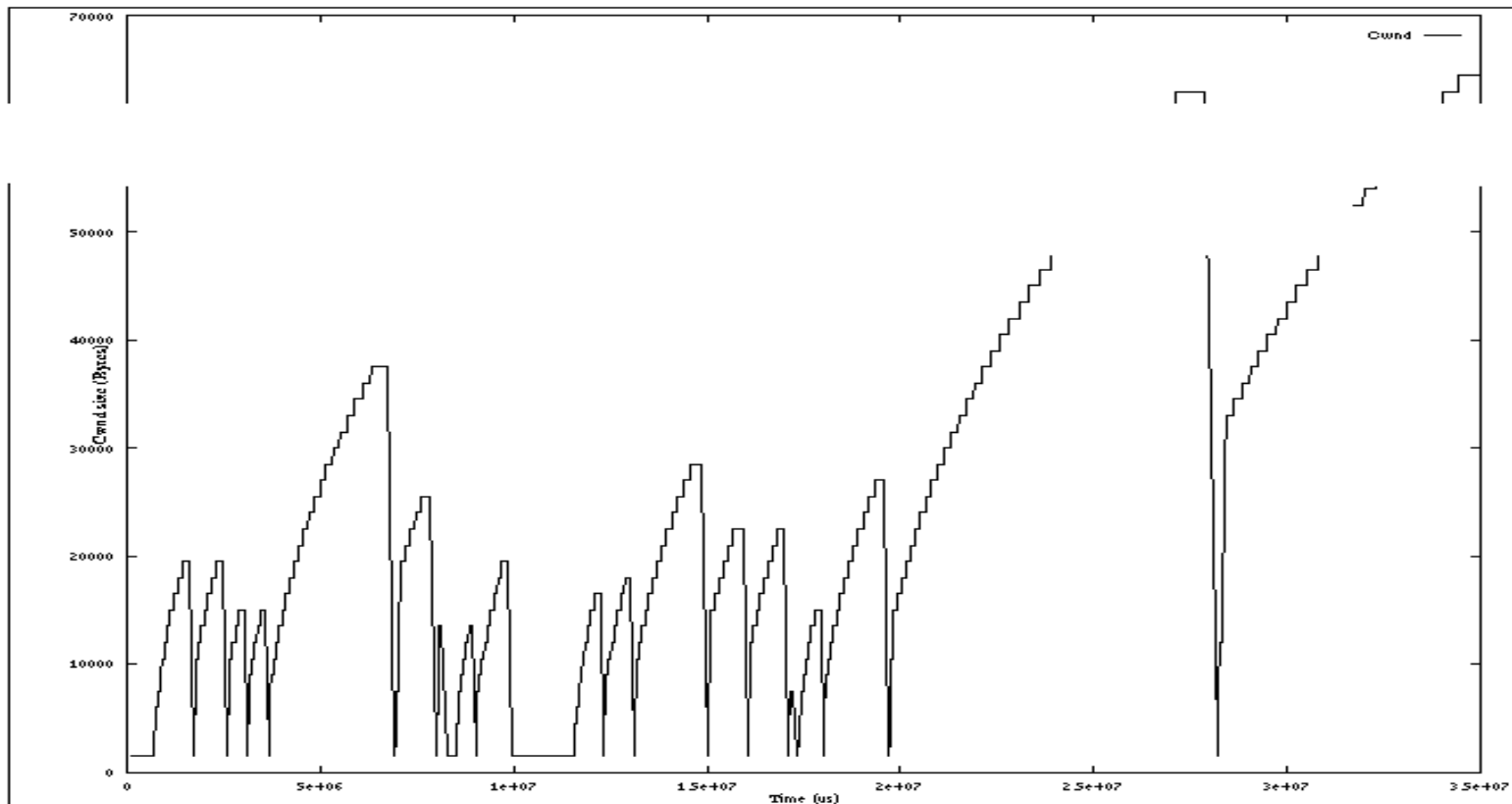
- ssthresh is set to half the current size of the congestion window:

$$\text{ssthresh} = \lfloor \text{cwnd} / 2 \rfloor$$

- and slow-start is entered

Slow Start / Congestion Avoidance

- A typical plot of cwnd for a TCP connection (MSS = 1500 bytes) with TCP Tahoe:



TCP Error Control

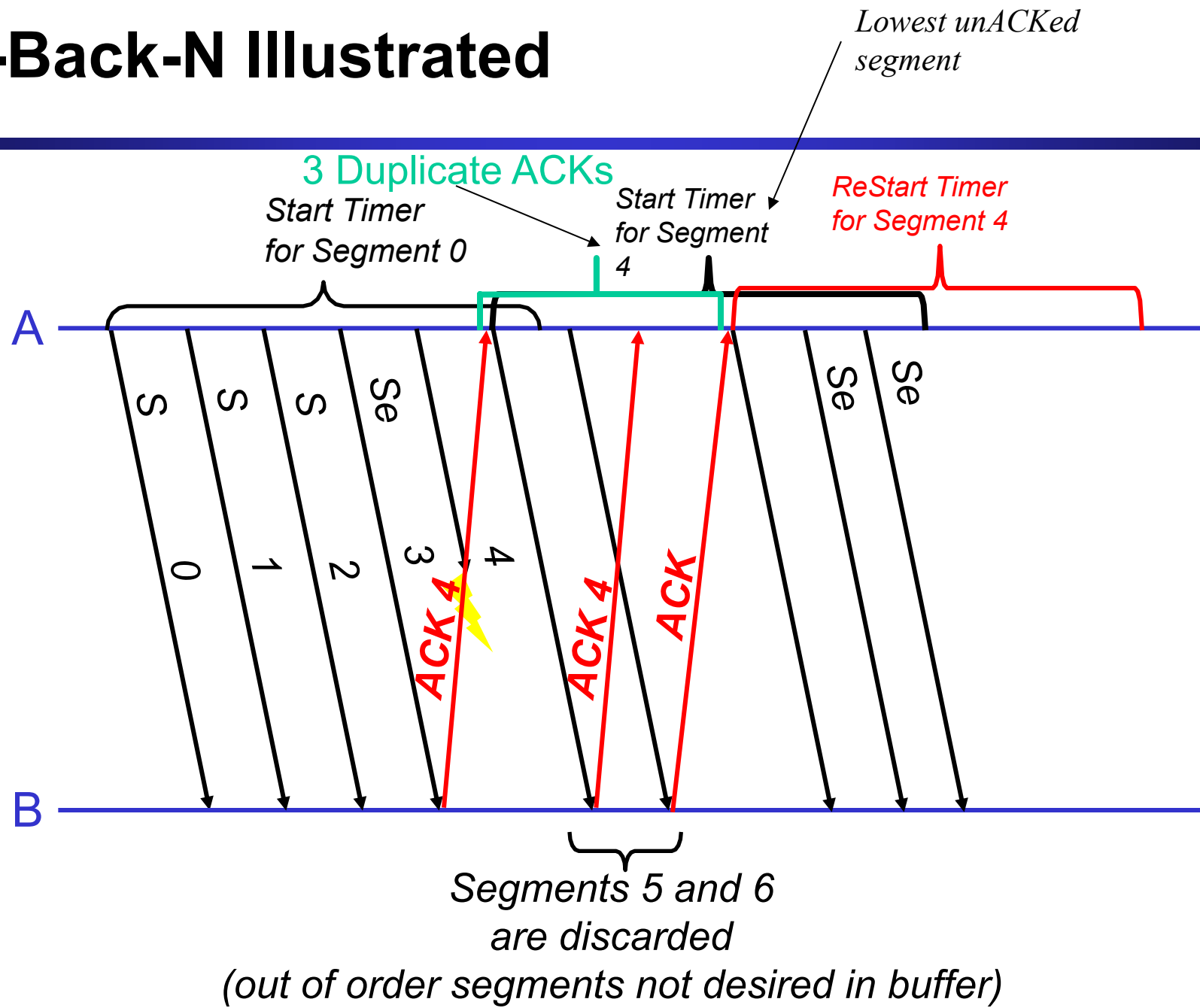
Go Back N Error Control

- Assume a sliding window protocol with a transmit (send) window size of **N**
- The **sender** transmits bytes in its buffer up to the allowed value - **Send**
window size = N
- The **receiver** will discard that do not have the Seq.
- k and send all
- duplicate A k and send all
packets again starting from Seq. No = Ack No. of the last ACK it received
from the receiver

Error Control in TCP

- TCP implements a variation of the **Go-back-N** retransmission scheme
- That means that if a segment is lost, all subsequent segments are dropped by the receiver as to retransmit all segments the tra
SeqNo. of the missing segment → Duplicate ACKs (DUPAck) peating the
- TCP **Tahoe** reacts after receiving the 3rd DUPAck by retransmitting **that** segment and **all** subsequent segments.
- TCP uses cumulative ACKs. Several received segments can be ACKed by one ACK by sending an ACKNo that corresponds to the last correctly received consecutive segment.

Go-Back-N Illustrated



TCP Tahoe

- TCP couples **error control** and **congestion control** (i.e., it assumes that errors (losses) are caused by congestion)
- Two conditions will cause TCP to go into Slow Start Mode (from any state):
 - n
 - When 3 t is lost,
retransmit and go back into SLOW START mode
- Note that when a segment is received by TCP and an error is detected the receiver will discard the segment and send a DUPAck repeating the last ACK No. it sent out.
- DUPAcks are also sent when segments are received out of order.

TCP Reno

- TCP Reno allows accelerated retransmissions when you are in **Congestion Avoidance Mode - Fast Retransmit**
 - If the sender receives 3 Duplicate ACKs it retransmits the assumed lost segment immediately
- - After a fast retransmit, the sender enters **Fast Recovery Mode** using $ssthresh = new$ $ssthresh$
- When in **Fast Recovery mode**:
 - if a timeout occurs for retransmitted segment, you go into SLOW START mode with $ssthresh = CWND/2$, and $CWND = 1$.
 - if you receive an ACK for retransmitted segment you go back into **Congestion Avoidance**

TCP Send Window

- In TCP we have two parameters to keep track of when deciding how much data may be transmitted:
- Readiness of receiver to accept sent data – **flow control window**
- network c
- **A sender's Send** window is therefore the min of receiver's flow control window and the congestion window

Send Window = MIN (flow control window, congestion window)

- **flow control window** is advertised by the **receiver**
- **congestion window** is computed/set at **sender** and adjusted based upon feedback from the network

Retransmission Timer when Errors Occur

- First timeout timer for a segment is set to Estimated RTT (round trip time)
- The interval between retransmission attempts (s) inc ret values are **not** calculation.
- Time between retransmissions is doubled each time (**Exponential Backoff Algorithm**)
- Timer is not increased beyond 64
- TCP gives up after 13th attempt
1, 2, 4, 8, 16, 32, 64, 64, 64, 64, 64, 64, 64

